.NET Security & Reliability(Console/Web app	Understanding security in .NET applications
	Common security practices (authentication, authorization, encryption)
	Secure coding practices
	Using .NET libraries for encryption, secure communication, and secure storage
	Building Reliable Applications
	Designing for reliability
	Exception handling strategies
	Error Handling & Logging
	Implementing error handling best practices
	Logging errors and monitoring application health

Learning Objectives

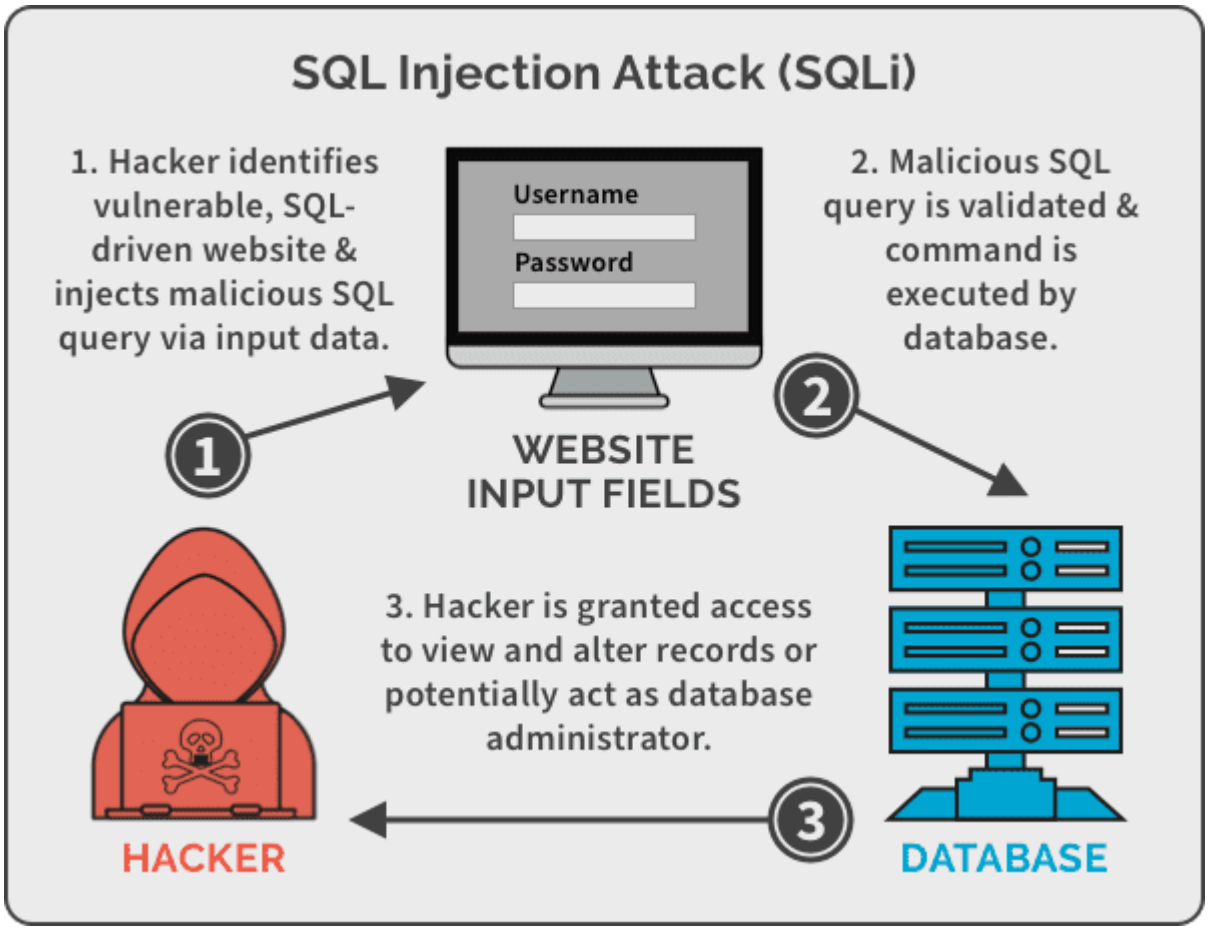
- Understand application security fundamentals
- Identify common vulnerabilities
- Learn security best practices

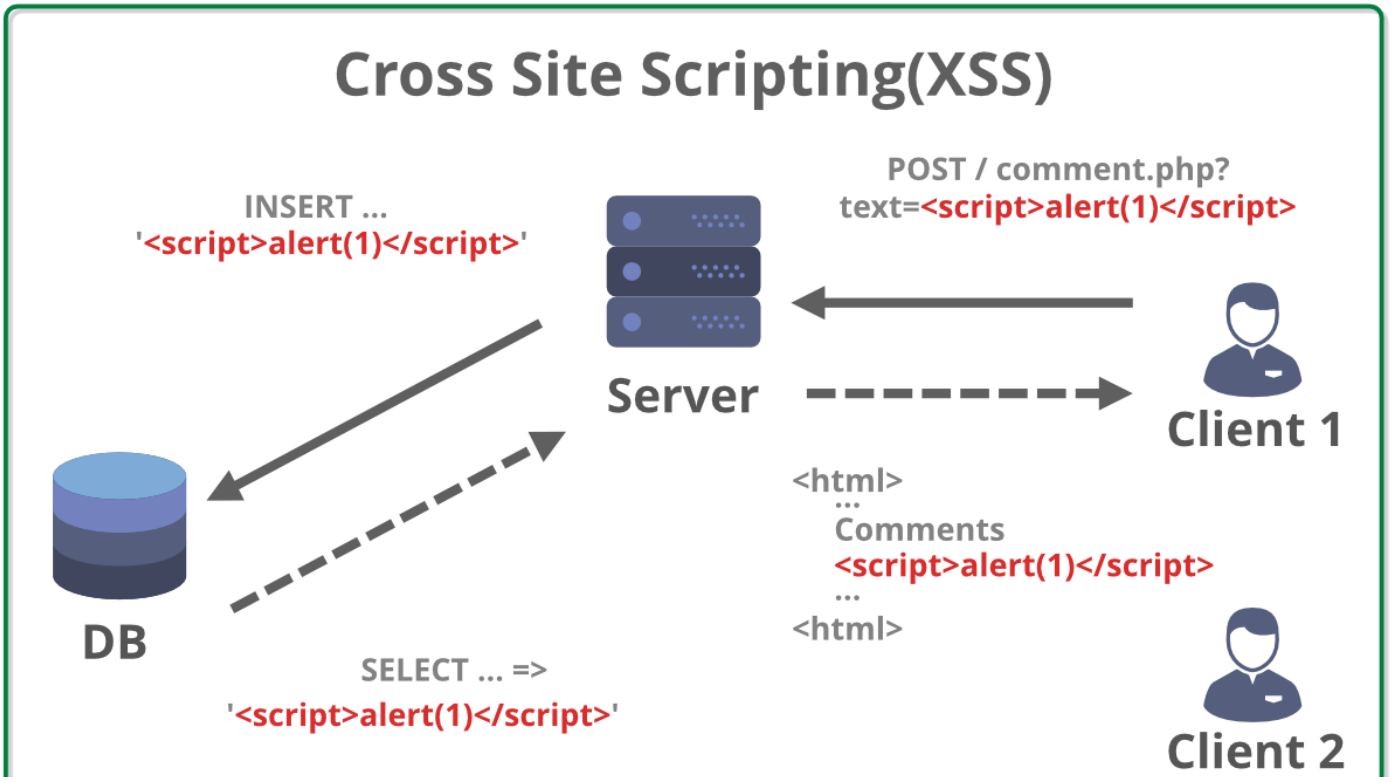
Key Security Concepts

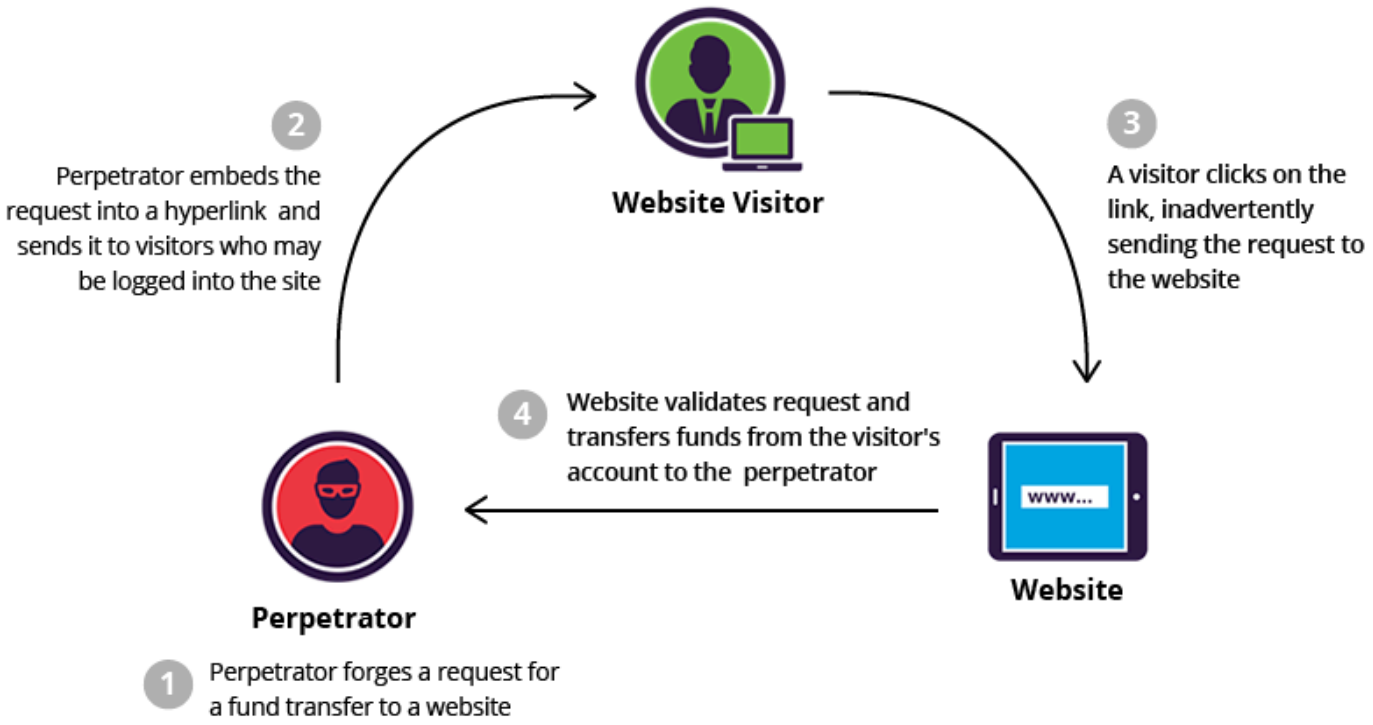
- Authentication (Who are you?)
- Authorization (What can you access?)
- Confidentiality (Protect data)
- Integrity (Prevent data tampering)

Common Vulnerabilities

- SQL Injection
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgery (CSRF)







Secure Coding Guidelines

- Validate all inputs
- Use parameterized queries
- Avoid hardcoded credentials
- Use HTTPS

User Story ID	User Story	Task Description	Expected Outcome	Concept Covered
US1	As a developer, I want to hash a user's password so that it is stored securely	Implement HashPassword() method using SHA256 and take user input from console	Password is converted into a hashed string (not readable)	Encryption using System.Security.Cryptography
US2	As a developer, I want to store the password securely so that sensitive data is protected	Store the hashed password in a variable or file instead of plain text	Password is stored in encrypted format	Secure Storage
US3	As a developer, I want to validate user login securely so that only authorized users can access the system	Compare hashed input password with stored hashed password	Login success/failure message displayed correctly	Authentication + Hash Comparison
US4	As a developer, I want to ensure secure communication so that user data is protected during transmission	Discuss and document why HTTPS is required and how SSL certificates work	Learners explain importance of HTTPS and secure communication	Secure Communication

US5	As a developer, I want to follow secure coding practices so that my application is safe from attacks	Identify and avoid storing plain passwords and hardcoded credentials	Learners list best practices for secure coding	Secure Coding Practices

Demo 1: Implementing a secure login

- Take username/password input
- Hash password
- Compare with stored hash

Demo 2: Demo 2: Role-Based Authorization

- Admin
- Guest

Demo 3: Password encryption

- SHA256

Demo 4: Exception Handling and Logging

Mini Project (Console-Based)

Secure Login & Logging System

Features:

- User registration (store hashed password)
- Login validation
- Role-based access
- Error handling
- Logging to file

Step 1: Creating a console/web application using CLI dotnet new console
Step 2: Understanding the project structure dot new webapp
Step 3: Adding custom middleware to understand Request -> Middleware -> response
Step 4: Serving static files (wwwroot folder) here we can add any index.html file and also adding “app.Usestaticfile()” in program.cs
Step 5: Creating a razor page with Razor Syntax and Basics
Step 6: Here we can apply page model or form handling

Stage	Concept
Console App	.NET basics
Web App	ASP.NET Core
Middleware	Request pipeline
Static Files	Frontend assets
Razor Pages	UI structure
PageModel	Backend logic
Forms	User interaction

We will be able to this follow :

1. Create a web app in .NET
2. Understand request flow
3. Build UI with Razor Pages
4. Handle user input